# Performance Testing & JMeter — Complete Notes

one clear explanation per concept — definition, analogy, formula, example, and tips together

## Contents


- Chapter 1 — Core Performance Concepts
- Chapter 2 — JMeter Foundations for Beginners

# Chapter 1 — Core Performance Concepts

definition, analogy, formula, and example — together, once, per concept

## 1 What a Server Does, and What JMeter Does

**Definition:** A server is a program/machine that receives a request over the network, processes it, and returns a response. JMeter is an open-source load testing tool that simulates many concurrent virtual users sending such requests, so teams can measure a system's performance and stability before real users do.

**Analogy:** A server works like a kitchen — it receives an order, prepares it, and sends the finished dish back to the table (User = customer, Network = waiter, Server = kitchen, Request = the order, Response = the food). JMeter is like deliberately sending in a large number of simulated customers ahead of a real Friday-night rush, to see whether the kitchen can keep up before it actually happens.




**WATCH OUT**

If too many customers order at once and there's only one chef, everyone waits longer — exactly what happens to a server under heavy concurrent load.

## 2 Little's Law

**Definition:** A queueing-theory formula stating that the average number of requests/users present in a system equals the arrival rate (throughput) multiplied by the average time each request spends in the system (response time). In performance testing, it's mainly used in reverse — to calculate how many virtual users are needed in JMeter to sustain a target throughput.

**Analogy:** The number of people inside a coffee shop at any moment equals how fast people arrive, multiplied by how long each person stays before leaving.

### $L = \lambda \times W$ (Concurrent Users = Throughput × Response Time)

Throughput: 50 TPS × Resp. Time: 2 sec = 100 users needed concurrently

**Example:** To sustain 50 transactions/sec where each transaction takes 2 seconds, required concurrent users = 50 × 2 = **100 users**.

**WATCH OUT**

If we add think time (say 3 sec pause), the formula becomes Users = Throughput × (Response Time + Think Time) → 50 × (2+3) = 250 users. Ignoring think time leads to under-estimating the number of users needed.

---

## 3 Throughput, TPS & Hits per Second

**Definition:**

- **Throughput** — number of individual HTTP requests completed per second.
- **TPS (Transactions Per Second)** — number of complete business flows (e.g. Login → Dashboard → Profile), grouped via a Transaction Controller, completed per second.
- **Hits/sec** — number of individual HTTP calls hitting the server per second, including embedded resources like images, CSS, and JS.

**Analogy:**

- **Throughput** — vehicles passing a toll booth per minute
- **TPS** — full trips completed end-to-end per minute
- **Hits/sec** — every individual toll-booth scan, including every axle counted separately — always the largest number of the three

Throughput — 1 HTTP request completed

TPS — 1 full business flow (login → dashboard → profile)

Hits/sec — every image, script, and static file counted individually

| Metric | Formula | Example |
|---|---|---|
| Throughput | Requests ÷ Time | 6000 requests / 120 sec = 50 req/sec |
| TPS | Business flows ÷ Time | 30 logins / 60 sec = 0.5 TPS |
| Hits/sec | Total HTTP calls ÷ Time | 10 users × 6 calls/page = 60 hits/sec |

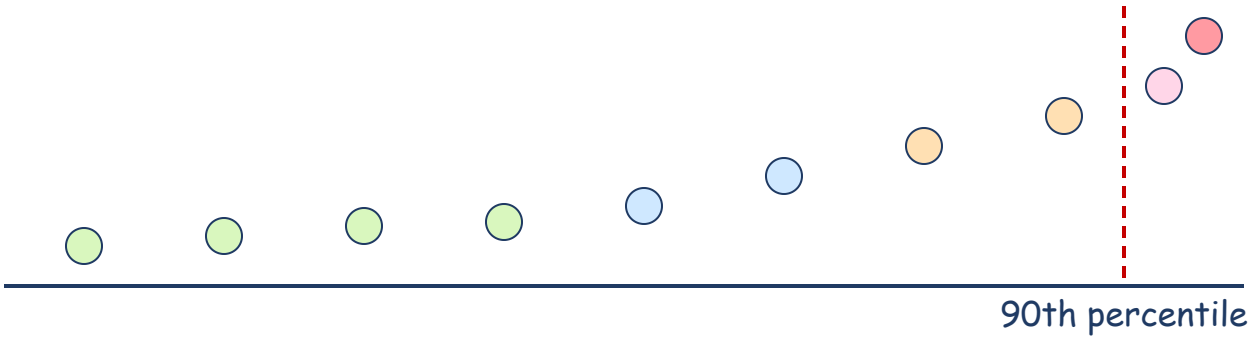
**TIP**

Throughput reported by JMeter is client-side. Always cross-check against server-side APM tools (New Relic, Dynatrace) or access logs. Also clarify with stakeholders whether an NFR document's "TPS" means business transactions or raw HTTP requests.

## 4 Percentiles — 90th, 95th, 99th

**Definition:** The Nth percentile is the response time value under which N% of all requests completed. Average response time can be misleading because it is skewed or masked by outliers; percentiles reveal the experience of the slowest fraction of users.

**Analogy:** If a class runs a race, the average finish time can look fine even if a few students stumbled and took much longer. The 90th percentile tells you the time by which 90% of the class had finished — it does not let those outliers hide.



| Metric | Sample set (ms): 100,120,130,140,150,900,950,1000,1100,5000 |
|---|---|
| Average | ~959 ms (misleading) |
| Median (50th) | 525 ms |
| 90th percentile | ~1100 ms |
| 99th percentile | ~5000 ms |

**KEY TAKEAWAY**

Always report and set SLAs using the 90th/95th/99th percentile as pass/fail criteria — never the average response time.

---

## 5 Response Time vs Latency

> **Definition:** Latency is the time from when a request is sent to when the first byte of the response is received — mostly network delay and server queueing time. Response Time (Sample Time) is latency plus the time taken to receive the complete response body.

**TIP**

In JMeter's View Results Tree, check "Connect Time", "Latency" and "Load Time" separately to diagnose whether a slowdown is network-related or data-transfer related.

---

## 6 Think Time & Pacing

**Definition:** Think Time is a deliberate pause inserted between requests in a script, via JMeter Timers, to simulate the delay a real user takes between actions. Pacing controls the gap between the end of one full iteration of a user's script and the start of the next.

**Analogy:** A customer doesn't place their next order the instant their food arrives — they eat, look at the menu, and decide. That natural pause is think time.

**TIP**

Script think time based on real user behaviour analytics where available, rather than guessing arbitrary values.

---

## 7 Ramp-Up, Steady State & Ramp-Down

**Definition:** The three phases of a load test: **Ramp-Up** gradually starts virtual users instead of launching them all instantly; **Steady State (Hold Time)** sustains the full target load for a fixed duration, during which actual measurements should be recorded; **Ramp-Down** gradually stops users at the end instead of disconnecting everyone at once.

**Analogy:** Like guests at a gathering — they arrive gradually (Ramp-Up), the event is in full swing while everyone is present and this is when you take stock of how it's going (Steady State), and then they leave gradually rather than all walking out at once (Ramp-Down).

Ramp-up time = Number of Threads ÷ Threads started per second




**Example:** 100 users ramp up over 100 sec (roughly 1 new user/sec), hold steady for 30 minutes, then ramp down over 60 sec. JMeter's standard Thread Group has no dedicated ramp-down field — this is commonly achieved using the **Ultimate Thread Group** or **Concurrency Thread Group** plugins.

**WATCH OUT**

A ramp-up that's too short causes an artificial spike unrelated to real traffic, and an abrupt end without ramp-down can kill connections mid-request, inflating error counts with test-harness artifacts rather than real application errors. Always measure results during Steady State only.

## 8 Concurrency vs Load

**Definition:** Concurrency is the number of virtual users/threads JMeter runs in parallel. Load is the actual throughput/TPS that results from that concurrency — the two are not the same thing.

KEY TAKEAWAY

Beyond a certain point, more threads just increase queueing and response time — the server has hit capacity. Plot "Active Threads Over Time" alongside Throughput and Response Time to spot the saturation point.

---

## 9 Types of Performance Tests

**Definition:**

- **Load Testing** — validates behaviour under expected, normal production load.
- **Stress Testing** — gradually increases load beyond expected levels until the system starts failing, to find the breaking point.
- **Soak / Endurance Testing** — runs moderate load for a long duration to catch memory leaks and gradual degradation.
- **Spike Testing** — applies a sudden, sharp increase in load to test unexpected bursts.
- **Scalability Testing** — checks whether performance improves proportionally when more resources are added.

| Test Type | Example |
|---|---|
| Load Test | 500 users, 1 hr |
| Stress Test | Increase users until failure |
| Soak Test | 200 users, 8–12 hrs |
| Spike Test | 50 → 1000 users in 30 sec |
| Scalability Test | Compare 2 vs 4 app servers |

---

## 10 Error Rate

> **Definition:** The percentage of requests that failed out of the total sent.

### Error % = (Failed Requests ÷ Total Requests) × 100

**Example:** 50 failed out of 5000 total = 1% error rate.

**WATCH OUT**

Common causes: connection pool exhaustion, request timeouts, server-side 5xx errors from overload, or incorrect assertions in the script itself. As a general benchmark, error rate under expected load should stay below 1%.

---

## 11 Correlation & Parameterization

**Definition:** Correlation is extracting a dynamic value from one server response (session token, CSRF token, view-state) and reusing it in subsequent requests — done via extractors (Regular Expression, JSON, CSS/JQuery, or Boundary Extractor). Parameterization is feeding different input data to each iteration/user via CSV Data Set Config, avoiding caching artifacts and simulating varied, realistic user behaviour.

**Example:** Regular Expression Extractor — Reference Name = csrfToken, Regex = `name="csrf_token" value="(.+?)"`, reused as `${csrfToken}`.

**WATCH OUT**

Recorded scripts with hardcoded session tokens/IDs will fail on replay because these values change per session — always correlate dynamic values.

---

## 12 Caching

**Definition:** Storing a previously computed or retrieved result so repeated requests for the same data can be served faster, without redoing the underlying computation or database lookup.

**Analogy:** If a hundred customers order the same dish, a kitchen that keeps a few pre-made portions ready is far faster than cooking each one individually from scratch.

**WATCH OUT**

If all virtual users in a test request identical data, caching can make the system look unrealistically fast. Parameterize inputs for an accurate picture.

---

## 13 Scaling

**Definition:** Increasing a system's capacity to handle more load, either by adding more power to an existing machine (**Vertical Scaling** — more CPU/RAM on the same machine) or by adding more machines to share the load (**Horizontal Scaling** — more server instances).

**Analogy:** Vertical scaling is a bigger stove for the same chef; horizontal scaling is hiring more chefs.

**TIP**

A shared dependency, like a single database, can limit gains regardless of how many additional servers are added — scalability testing verifies this.

---

## 14 Connection Pool

**Definition:** A fixed set of pre-established connections (typically to a database) that a server reuses across requests instead of opening a new connection each time. Once all connections are in use, further requests must wait for one to free up.

**Analogy:** A restaurant with a fixed number of tables — once all tables are occupied, new customers wait outside even though the kitchen may not be busy.

WATCH OUT

A server with low CPU/memory usage can still be slow because it has run out of available connections in its pool — a common, easy-to-miss bottleneck.

## 15 Warm-Up Period

**Definition:** A period of light traffic run before the actual test to let caches populate and runtime optimizations (such as JIT compilation) take effect, so the real test measures steady operating performance rather than cold-start behaviour.

**Analogy:** A cold oven takes longer to cook the first dish than one that's already been running for a while.

## 16 SLA / NFR

**Definition:** A Service Level Agreement (SLA) or Non-Functional Requirement (NFR) is a documented, measurable performance standard a system must meet — e.g. response time thresholds, throughput targets, and acceptable error rates — verified through performance testing before release.

**Analogy:** A restaurant promising "your food will arrive within 15 minutes, 95% of the time" is making the same kind of commitment as an SLA stating "95% of homepage requests must load in under 2 seconds, with error rate below 1%."

# 17 Bottleneck

> **Definition:** The single most constrained component in a system — whether a database, external API, memory, thread pool, or connection pool — that limits overall performance regardless of how well-provisioned the rest of the system is.

> **Analogy:** A large hall full of people can only exit as fast as its narrowest doorway allows — no matter how big the hall itself is.

1. Active threads rising, throughput flat/dropping → server saturation
2. Response time climbs then spikes suddenly → resource exhaustion (CPU/DB pool/threads)
3. Correlate with server metrics: CPU%, memory, GC pauses, DB query time
4. Watch error rate rise right at the saturation point

**KEY TAKEAWAY**

Performance engineering is largely about finding this constrained point — improving anything else won't help until the actual bottleneck is addressed.

---

# 18 Monitoring

> **Definition:** The practice of observing server-side resource metrics (CPU, memory, database query time, error logs) during a load test, in parallel with JMeter's client-side results, to identify the root cause of performance issues rather than just their symptoms.

**Analogy:** Watching a kitchen through a camera while orders come in helps explain why service slowed down, rather than only noticing that it did.

**TIP**

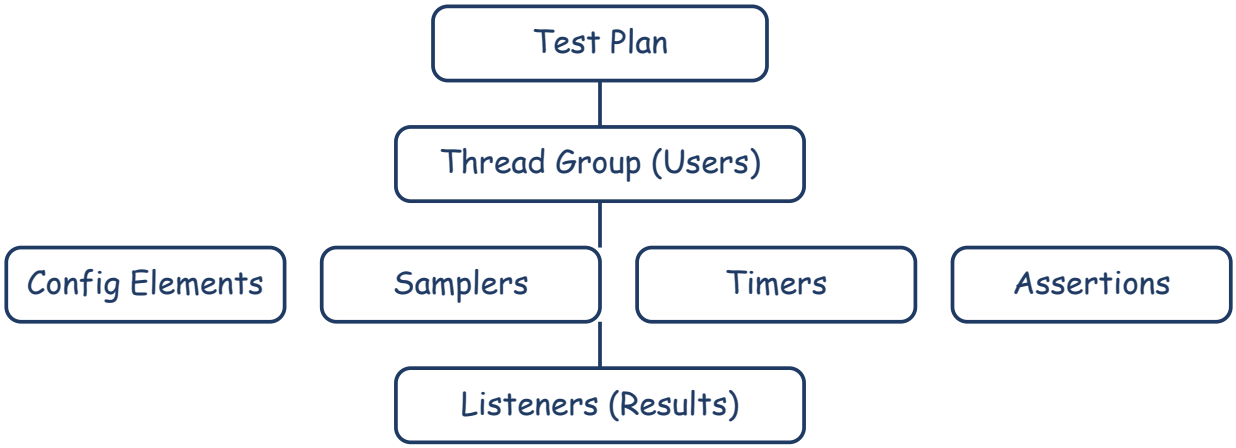
Tools like Grafana, Dynatrace, or New Relic provide CPU%, memory, DB query time, and error logs during the test — correlate these with JMeter's results for root-cause analysis.

# Chapter 2 — JMeter Foundations for Beginners

the building blocks to know before writing your first script

## ⑲ JMeter Architecture Basics

- **Test Plan** — the root container for everything in your test.
- **Thread Group** — defines virtual users: how many, how fast to start them (ramp-up), and how many times to loop.
- **Samplers** — the actual requests sent to the server, e.g. HTTP Request, JDBC Request, FTP Request.
- **Logic Controllers** — control the flow/order of samplers, e.g. Loop Controller, If Controller, Transaction Controller.
- **Config Elements** — shared configuration used by samplers, e.g. HTTP Header Manager, CSV Data Set Config, HTTP Cookie Manager.
- **Timers** — add delays/pauses between requests (think time).
- **Assertions** — validate that the response is actually correct.
- **Listeners** — collect and display results.




**WATCH OUT**

Config Elements, Timers and Assertions placed inside a Thread Group apply to everything below them in that branch — placement in the tree changes behaviour.

---

## 20 Assertions

- **Response Assertion** — checks the response body/headers/code for expected text.
- **Duration Assertion** — fails the sample if response time exceeds a defined threshold.
- **Size Assertion** — checks response size in bytes.
- **JSON Assertion** — validates specific fields/values inside a JSON response.

**Example:** After a login request, add a Response Assertion checking the body contains "Logout" — catches a silent login failure that still returns HTTP 200.

**TIP**

Add at least one assertion per critical business transaction — relying only on HTTP status codes gives a false sense of confidence.

---

## 21 CSV Data Set Config — Sharing Modes

> **Example CSV (users.csv):**
> `username,password`
> `user1,pass1`
> `user2,pass2`
> Referenced as `${username}` and `${password}`.

- **All threads** — every thread pulls from a shared pointer; rows distributed once across all users (default).
- **Current thread group** — data shared only within that specific Thread Group.
- **Current thread** — each thread reads independently from the start; rarely what you want for unique data.

**WATCH OUT**

Not having enough rows for the number of users/iterations causes JMeter to recycle data or fail requests partway through, depending on "Recycle on EOF".

---

## ㉒ Workload Models — Open vs Closed System

**Closed System** (fixed thread count) — a fixed number of virtual users loop continuously, each waiting for their previous request to finish before sending the next. If the server slows down, the arrival rate of new requests drops too.

**Open System** (fixed arrival rate) — new requests arrive at a fixed rate regardless of server response speed, like real internet traffic. JMeter's standard Thread Group is a closed model; an open model needs the Arrivals Thread Group / Concurrency Thread Group plugins, or a Throughput Shaping Timer.

> WATCH OUT
>
> Choosing the wrong workload model is a common reason test results don't match real production incidents.

---

## ㉓ Distributed Testing (Master-Slave)

A single JMeter machine has limits (CPU, memory, network). For large-scale load, JMeter supports distributed testing — one machine as the Controller/Master, several as Load Generators/Slaves executing the same test plan in parallel, aggregating results back to the master.

> **TIP**
>
> Always monitor CPU/memory on the load generator machines themselves — if they're maxed out, results reflect the load generator's limits, not the application's.

---

## 24 GUI Mode vs Non-GUI (CLI) Mode

GUI Mode is for building and debugging scripts on a small scale. Non-GUI Mode runs the test from the command line, using far less CPU/memory — this is the mode for actual load generation.

```
jmeter -n -t test_plan.jmx -l results.jtl -e -o report_output
```

> **WATCH OUT**
>
> Never generate real load through the GUI. Use it only to build and debug scripts with a couple of users, then switch to CLI mode for the actual run.

---

## 25 Baseline vs Benchmark Testing

**Baseline Testing** — running a test against a known-stable version to record reference performance. **Benchmark Testing** — running the same test against a new build and comparing against the baseline to detect regressions.

**TIP**

Keep test conditions identical between baseline and benchmark runs — same load, environment, and data set — otherwise the comparison is meaningless.

---

## 26 Common Beginner Mistakes

1. **Leaving heavy Listeners enabled during load runs** — View Results Tree and Graph Results consume CPU/memory on the JMeter machine and can crash the test client before the server is even stressed.

2. **No assertions** — relying purely on HTTP status codes can hide real functional failures.

3. **Ignoring correlation** — hardcoded session tokens/IDs fail on replay because values change per session.

4. **No think time** — back-to-back requests create unrealistic load.

5. **Not increasing JMeter's heap size** — default JVM heap is small; larger tests need it increased or you'll hit OutOfMemoryError.

6. **Unrealistic or too little test data** — reusing the same account/product ID triggers caching effects that make the app look faster than reality.

7. **Not accounting for ramp-up/down in analysis** — measure only during Steady State.

8. **Load generator and monitoring on the same constrained machine** — this can distort both the applied load and observed metrics.

End of notes — one definition, one analogy, one set of tips per concept.